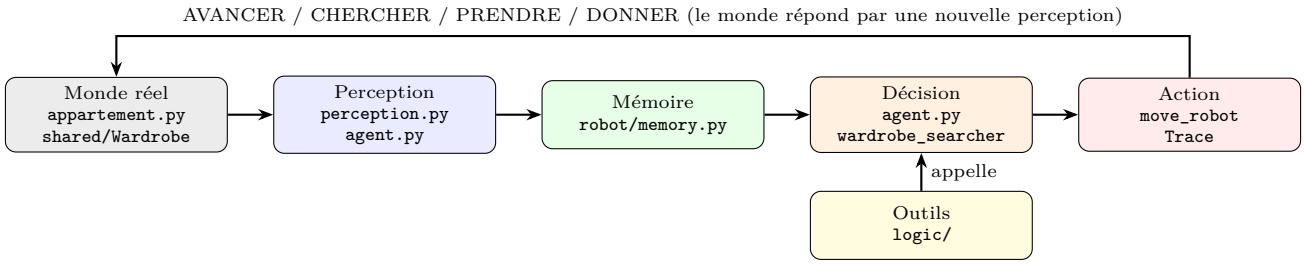
Le robot de réconfort — Rapport

HAI716 | ABDEL SATTAR, GUIMARAES CAIXETA SILVA

1. Architecture



Les quatre rôles attendus et ce que fait notre code. Pour chaque rôle, nous rappelons ce qui est attendu, puis où et comment notre code le réalise.

1. **Perception.** Attendu : recevoir le type des quatre cases voisines et, devant l'armoire, le contenu du casier courant, sans rien décider.
   Chez nous : l'agent n'accède jamais à la grille. Il demande à `Appartement` (`appartement.py`) de percevoir son environnement avec `perceive()`. `Appartement` lit la vraie grille et ne renvoie que les quatre observations locales (mur, libre, armoire, dictionnaire ou résident). `perception.py` (`valider_perception`) vérifie ensuite que les quatre directions sont présentes et que chaque type est connu. Le contenu du casier courant est lu par `Wardrobe.check_object()`, appelé depuis `look_content` dans `wardrobe_searcher.py`. Cette partie est séparée du reste.
2. **Mémoire.** Attendu : conserver ce que le robot sait : carte mentale, position, position du sélecteur, objet porté, étape de la demande en cours, casiers déjà vus.
   Chez nous : le dossier `robot/` conserve la carte mentale (`Memory.map`, mise à jour par `update_map`), les casiers vus (`WardrobeMemory`, états inconnu, vide ou garni), les mots déjà consultés (`DictionaryMemory`), la position du sélecteur et l'objet porté. Deux éléments attendus manquent : la position du robot est lue dans le simulateur (`Appartement.robot_position`), et l'étape de la demande n'existe que dans le déroulement de la boucle de `agent.py`, pas comme donnée.
3. **Décision.** Attendu : lire la mémoire et renvoyer la prochaine action, par exemple (`AVANCER`, `N`), sans rien modifier.
   Chez nous, la décision n'est pas dans une seule fonction : elle se prend à trois endroits.
   (a) Déplacement dans l'appartement, dans `navigate_to` : `shortest_path` et `adjacent_goals` (`logic/bfs.py`) calculent un chemin sur la carte mentale, `best_frontier` choisit où explorer quand aucun chemin n'est connu, et `to_directions` en tire la direction du prochain pas. Ces fonctions ne modifient pas la mémoire, comme demandé.
   (b) Choix dans l'armoire, dans `WardrobeSearcher` : `target_box` retient le prochain casier de la liste ordonnée de `EmotionWheel.candidates` qui n'est pas connu vide, et `get_route` en déduit la suite de mouvements du sélecteur.
   (c) Enchaînement des étapes d'une demande, dans la boucle principale de `agent.py` : aller au dictionnaire (sauf si les mots du message sont déjà en mémoire), puis à l'armoire, puis chez le résident, ou abandonner la demande avec un motif d'échec.
4. **Action.** Attendu : exécuter l'action (nouvelle position, nouveau casier, objet pris ou donné) et écrire le pas dans la trace, sans rien choisir.
   Chez nous, l'action est aussi répartie à trois endroits.
   (a) `move_robot` demande le déplacement à `Appartement` (`move`), qui vérifie la case sur la vraie grille, exécute le mouvement et indique s'il a réussi, puis écrit le pas `AVANCER`.
   (b) `WardrobeSearcher.search` déplace le sélecteur (`Wardrobe.move_route`), prend l'objet (`take_object`) et écrit les pas `CHERCHER` et `PRENDRE`.
   (c) La boucle principale de `agent.py` écrit directement les pas `CONSULTER` et `DONNER`.

Le rôle de `navigate_to`, et comment il mélange les quatre parties. Cette fonction amène le robot sur une case adjacente à une cible (dictionnaire, armoire ou résident). Elle est appelée trois fois par demande, et chaque tour de sa boucle enchaîne les quatre rôles :

1. Perception et mémoire : `receive_perception` appelle `Appartement.perceive`, valide le résultat, puis `memory.update_map` inscrit les quatre cases dans la carte mentale.
2. Décision, chemin connu : `adjacent_goals` puis `shortest_path` depuis la position courante. Si le robot est déjà adjacent à la cible, la fonction s'arrête avec succès.
3. Décision, exploration : sans chemin connu, `best_frontier` choisit une frontière à explorer. S'il n'en reste aucune, la fonction conclut que la cible est inaccessible et s'arrête avec un échec.

4. Action : `move_robot` exécute la direction du premier pas du chemin, puis écrit le pas dans la trace. Au tour suivant, tout est recalculé.

Exemple sur `appartement_01` : en $[6, 6]$, la perception signale un mur à l'est (étape 1), le chemin est recalculé et passe par le nord (étapes 2 et 3), et le robot avance vers le nord (étape 4).

Le mélange vient de ce que la décision n'est pas rendue comme une action à un exécutant séparé : elle est calculée puis exécutée dans la même boucle. Trois conséquences : la fonction lit la position du robot dans le simulateur ; elle est définie à l'intérieur de `main`, donc elle ne se teste pas sans lancer toute la simulation ; et `move_robot` affiche aussi la carte mentale, ce qui n'est pas son rôle.

Organisation en dossiers. Pour limiter ces mélanges, le code est rangé par rôle : `shared/` contient le monde et les types communs (le vrai contenu de l'armoire, `Map`, `Box`, `Emotion`), `robot/` contient tout ce que le robot sait, `logic/` contient les algorithmes sans état qui servent d'outils de décision, et `agent.py` contient le flux principal.

Justification des frontières.

1. L'armoire réelle (`shared/Wardrobe`) et sa représentation mentale (`WardrobeMemory`) partagent le type `Box` mais n'échangent rien directement : seule la lecture du casier courant fait passer l'information. Un casier ne peut donc être déclaré vide que s'il a été vu.
2. Seule la classe `Appartement` lit la grille réelle. L'agent ne lui adresse que deux demandes : percevoir (`perceive`) ou exécuter un déplacement (`move`). `Appartement` répond par les quatre observations locales, ou par la réussite ou l'échec du mouvement. Le BFS reçoit un prédicat `is_walkable` évalué sur la carte mentale et ne peut pas toucher la vraie grille : l'interdiction de lire la grille est respectée.
3. `logic/` ne dépend ni du simulateur ni de la mémoire : chaque règle imposée (lecture du dictionnaire, choix du casier de remplacement) se teste seule sur de petites données écrites à la main.

2. Stratégie de replanification

Notre robot ne répare pas son plan : à chaque tour il perçoit, met à jour la carte, puis recalcule tout depuis sa position et ne fait qu'un pas (`navigate_to`). Un mur découvert n'a donc pas besoin d'être détecté : le BFS suivant en tient compte.

Contrairement à l'approche qui suppose libre tout ce qui n'a pas encore été vu, les cases inconnues sont ici non traversables (`is_walkable` n'accepte que `.`). Si aucun chemin n'existe jusqu'à une case adjacente à la cible sur les cases connues, le robot passe en exploration : il liste les frontières (case libre connue ayant un voisin inconnu) et va vers celle qui minimise la distance parcourue plus la distance de Manhattan à la cible.

Coût. Soit $N = H \times W$. Un tour coûte $O(N)$ quand un chemin connu existe (un BFS). Sans chemin connu : $O(N)$ pour lister les frontières puis un BFS par frontière, soit $O(F \cdot N)$ avec $F \leq N$. Sur 15×21 c'est négligeable, mais cela deviendrait lourd sur de grandes cartes. Un seul BFS depuis le robot donnerait toutes les distances en $O(N)$.

Inaccessibilité. `best_frontier` renvoie `None` uniquement quand plus aucune case atteignable n'a de voisin inconnu : la région atteignable est alors entièrement connue et ne contient pas la cible, ce qui prouve qu'aucun chemin n'existe dans la réalité. Comme chaque arrivée sur une frontière révèle au moins une case, l'exploration termine. Le prix est d'explorer toute la région atteignable avant de conclure.

3. Critique de la règle de repli

La règle imposée remplace un casier vide par celui d'une émotion voisine sur la roue de Plutchik, jusqu'à deux crans. Or la roue décrit des ressemblances entre émotions, pas des compatibilités de réconfort. Pour une personne triste, les voisines immédiates sont le dégoût et la surprise : l'armoire standard répond alors avec des gants de ménage ou rien d'utile. Deux crans plus loin, une lampe torche ou un bouton d'appel (peur) peuvent même inquiéter. La règle est défendable comme choix déterministe et simple à vérifier automatiquement, mais pas comme choix de soin.

Épuisement des casiers. Chaque casier ne contient qu'un seul objet, et le prendre le vide pour toute la suite du scénario. Une livraison de repli a donc un coût caché : si un résident reçoit l'objet d'un casier voisin, qui ne satisfait qu'en partie son émotion, le casier est quand même vidé. Si un résident suivant exprime plus tard exactement l'émotion de ce casier, une demande qui aurait pu être pleinement satisfaite tombe à son tour sur un repli, ou échoue. La règle fait ainsi sacrifier une réponse exacte à venir pour une réponse approximative immédiate, sans jamais comparer les deux.

Proposition. Étiqueter les objets plutôt que les casiers. Dans le fichier d'armoire, chaque casier reçoit une liste `"reconforte"` (les émotions qu'il apaise : une couverture pour `tristesse` et `peur`) et un booléen `"neutre"` (tisane, plaid, coussin). Le repli devient :

1. même émotion, autre intensité (comme aujourd'hui) ;
2. un objet dont `reconforte` contient l'émotion demandée, trié par écart d'intensité ;
3. un objet `neutre` ;
4. sinon un échec explicite, plutôt qu'un objet inadapté.

Il faut modifier le format du fichier d'armoire (avec un nombre d'exemplaires par casier, décrémenté à chaque prise,

pour qu'un repli ne vide plus un casier utile aux demandes suivantes) et sa validation, la liste de candidats de `EmotionWheel` (qui dépendrait du contenu du fichier et non plus de la roue seule), et la trace (nouvelles valeurs possibles pour `repli`). Nous avons implémenté la règle imposée et non notre proposition, afin que nos livraisons restent comparables à celles de la correction.

4. Trois échecs de notre robot

Nous comptons en pas `AVANCER`, c'est-à-dire en déplacements d'une case du robot dans l'appartement. Le « chemin optimal » est le plus court chemin que suivrait un robot qui connaîtrait déjà le plan. Les mesures ont été obtenues en rejouant notre boucle de navigation (`navigate_to` et `logic/bfs.py`) sur la carte.

1. **Détour dû à l'heuristique d'exploration.**
   Situation : `appartement_01`, première demande, pour R1 en $[6, 11]$. Le robot part de $[6, 1]$, passe par le dictionnaire, puis par l'armoire ; nous regardons le dernier trajet, de $[6, 5]$ (devant l'armoire) jusqu'à une case voisine de R1.
   Ce que fait le robot :
   (a) À $[6, 5]$, il ne connaît que la moitié gauche de l'appartement et aucun chemin connu ne mène à R1. Il choisit la case à explorer qui minimise la distance déjà à parcourir plus la distance à vol d'oiseau jusqu'à R1 : la case $[6, 6]$, la plus à l'est.
   (b) En $[6, 6]$, il perçoit un mur en $[6, 7]$: la voie vers l'est est bouchée. Il a fait 1 pas dans le vide.
   (c) Il remonte en $[5, 6]$, perçoit un second mur en $[5, 7]$, puis repart vers l'ouest ($[5, 5]$ jusqu'à $[5, 1]$) et sort de la pièce par $[4, 1]$ et $[3, 1]$.
   (d) Il longe la ligne 2 vers l'est jusqu'à $[2, 11]$, descend par $[3, 11]$ et $[4, 11]$, et arrive en $[5, 11]$, voisin de R1.
   Mesure : 23 pas contre 21 pour le chemin optimal, soit 2 pas perdus (aller en $[6, 6]$ et revenir). Sur l'ensemble de la demande, 29 pas contre 27. Pour mesurer ce que coûte l'ignorance, nous avons aussi lancé ce même trajet depuis $[6, 5]$ avec une carte mentale vierge : 37 pas contre 21, soit 16 pas perdus, car le robot fait alors le tour de la pièce de gauche avant de trouver la sortie. Ce cas n'arrive pas ici, parce que les deux trajets précédents ont déjà révélé une partie de la pièce, mais il arriverait sur une grande carte dont le robot découvre une zone neuve.
   Cause : le score d'une case à explorer utilise la distance à vol d'oiseau, qui ignore les murs, et les cases inconnues sont non traversables : le robot n'a aucune information au-delà de ses quatre voisines. Il ne peut donc pas savoir que l'est est une impasse avant d'y être.
2. **Résident muré : exploration exhaustive et objet perdu.**
   Situation : variante de `appartement_01` où le passage $[4, 11]$ est muré. La pièce de R1 (lignes 5 à 7, colonnes 8 à 11) n'a plus aucune porte.
   Ce que fait le robot :
   (a) Il consulte le dictionnaire, va à l'armoire et prend l'objet, comme pour une demande normale.
   (b) Il part vers R1. Aucun chemin connu n'existe, il explore donc de proche en proche.
   (c) Il ne peut conclure que lorsque aucune case atteignable n'a plus de voisin inconnu. Comme une case dont les voisins ne sont pas tous connus ne s'élimine qu'en s'y rendant, il visite presque toute la partie atteignable : 39 cases différentes sur les 47 cases libres atteignables.
   (d) Il conclut en $[7, 2]$ que R1 est inaccessible, journalise l'échec, puis passe à la demande suivante. L'objet qu'il portait n'est jamais reposé dans son casier : il est perdu.
   Mesure : 48 pas `AVANCER` entre la prise de l'objet et la conclusion, tous inutiles puisque la demande échoue. Un robot qui connaîtrait le plan n'en ferait aucun.
   Conséquence : le casier d'où venait l'objet reste vide pour la suite du scénario. Quand une demande ultérieure le visera, le robot le trouvera vide et devra se rabattre sur un autre casier, ou échouer, alors que l'objet aurait pu servir à un autre résident.
   Cause : la conclusion est correcte mais son prix est l'exploration de toute la région atteignable ; de plus l'ordre des étapes (dictionnaire, armoire, résident) est figé, donc l'accessibilité du résident n'est jamais testée avant de prendre l'objet, et le robot ne sait pas reposer l'objet dans son casier quand la livraison échoue.
3. **Allers-retours dans l'armoire.** Situation : demande surprise/forte avec la colonne surprise entièrement vide, sélecteur en $[1, 0]$. Observation : 9 mouvements, dont deux repassages devant $[1, 3]$ et $[2, 3]$, déjà vus vides. Cause : `get_route` ne vise que le prochain candidat, toujours en vertical puis horizontal, sans regarder la suite de la liste. Nous mémorisons bien chaque casier croisé, mais ne choisissons pas l'itinéraire pour en croiser plusieurs utiles. Un robot qui descendrait la colonne dans l'autre sens économiserait des mouvements.